Lecture 05.11: Encapsulation in OOPs

📌 Executive Summary

Information hiding, public vs protected vs private attributes, name mangling mechanics, and property decorators (@property).

🔬 Theoretical Foundations & Principles

1. Access Control Conventions in Python

Unlike languages with rigid `public`, `protected`, and `private` keywords, Python uses lexical naming conventions:

- **Public (`attr`):** Accessible everywhere.
- **Protected (`_attr`):** Advisory convention signaling internal implementation detail; accessible by subclasses.
- **Private (`__attr`):** Triggers **Name Mangling** by the compiler.

2. CPython Name Mangling Architecture

Attributes with at least two leading underscores and at most one trailing underscore are rewritten at compile-time:

$$\text{Class.__attr} \implies \text{_Class__attr}$$

3. The Property Descriptor Pattern

Use `@property` and `@attr.setter` to expose attribute-like syntax while executing validation logic on writes.

💻 Production Implementation & Code

```
class Student:
    def __init__(self, name: str, roll_no: int, marks: float):
        self._name = name
```

```
        self._roll_no = roll_no
        self._marks = marks

    @property
    def marks(self) -> float:
        return self._marks

    @marks.setter
    def marks(self, value: float) -> None:
        if value < 0:
            raise ValueError(f"Marks cannot be negative, received {value}")
        self._marks = value

s = Student("Alice", 10, 85.0)
s.marks = 92.5  # Invokes setter
print(f"Student: {s._name} | Marks: {s.marks}")
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.